

CRM SaaS Platform — SRS Compliance Audit

Repository: `d:\nevacrm` · Branch: `dev` · Commit: `d3d9572`
Specification: `CRMSaaS SRD.txt` — Multi-Tenant CRM SaaS Platform, MVP Proposal & Product Reference Document
Date: 26 July 2026

Method. Full read of the SRS; inspection of the Prisma schema, all backend route / controller / use-case / domain files, and all frontend routes, pages and hooks; execution of both test suites and the frontend typechecker. Findings are evidence-based — no requirement is marked complete without a code reference.

1. Executive Summary

Overall completion: ~65% of MVP scope

Milestones 1–5 of the \$11 roadmap are substantially delivered. **Milestone 6 (Notifications) has not started.** Milestone 7 (deployment hardening) has not started.

The platform is not a mockup. The core transactional spine — auth → tenants → clients → appointments → inventory → quotations — is genuinely implemented end-to-end with a clean hexagonal architecture, real domain entities enforcing invariants, and 300+ passing tests. Where it falls short, it falls short in whole modules rather than in detail.

Major strengths

- Inventory & stock (\$6.4) is the strongest module** — multi-warehouse stock levels, transfers inside DB transactions, movement audit trail, derived availability, faceted search. Effectively complete.
- Quotations (\$6.5)** — the full `Draft` → `PendingApproval` → `Sent` → `Accepted/Rejected/Expired` lifecycle is enforced in the domain entity (`Quotation.ts:83–133`); the configurable approval toggle works; stock deduction on acceptance is transactional with pre-validation.
- Tenant isolation (\$4.5, \$8.5)** — every tenant-scoped router runs `authenticate` → `resolveTenant` , which rejects cross-tenant access. Every table below `Tenant` carries `tenantId` .
- Directory-based tenant routing (\$4.8)** — implemented exactly as specified in both Express and React Router.
- Architecture** — clean domain / application / infrastructure / interface separation; ports and adapters throughout; genuinely maintainable.

Major missing areas

Area	State
Notifications (\$6.6) — all three sub-features	0% — the module does not exist. No <code>Notification</code> model, no service, no scheduler. Email is wired only for invitations and password resets.
Settings (\$6.8) — company profile, localization, language	~15%. The UI exists but every field is <code>disabled</code> and unbacked. Only the quotation-approval toggle and logo/banner upload are real.
Multi-language (\$6.8, \$12.1)	0%. No i18n library, no translation layer, every string hard-coded — the exact opposite of the stated architectural requirement.
Reports (\$6.7)	~40%. Appointment statistics missing entirely; no client trend-over-time; no low-stock list; no per-business widget configuration.
Role-based dashboards (\$6.1)	~40%. Staff receive company-wide KPIs from the same unscoped endpoint as owners; several Staff and Super-Admin tiles are hard-coded fake numbers.

Area	State
Super Admin tenant management (\$5.1)	~20%. Read-only tenant list. No create, suspend or remove.

Overall readiness

Not production-ready. Two named MVP deliverables — notifications and multi-language support — are absent. The security posture has gaps that must close before a public deployment: a default JWT secret, the JWT duplicated into `localStorage`, no rate limiting, no `helmet`, and stack traces returned to clients on every 500.

Verification performed

- Backend `src/` suite: **212 / 212 tests pass** (42 suites).
- Backend `tests/` suite: **5 suites fail serially** (4 stale test doubles that no longer compile, 1 genuine assertion failure) and **13 suites fail in parallel** due to a shared test database with no per-worker isolation.
- Frontend: `tsc -b` clean; **90 / 90 tests pass** across 18 files.

2. Requirement-by-Requirement Checklist

✅ Fully Implemented ● Partially Implemented ❌ Missing ⚠ Implemented Differently

§4 — SaaS Architecture

Requirement	Status	Evidence	Notes
4.1 Multi-tenant, <code>tenantId</code> on every record	✅	<code>backend/prisma/schema.prisma</code> — every model below <code>Tenant</code> has <code>tenantId</code>	Composite indexes <code>@@index([tenantId, ...])</code> throughout.
4.2 Super Admin → Owner → Staff hierarchy	●	<code>auth/domain/enums/UserRole.ts</code> ; <code>middlewares/authorize.ts</code>	Three roles exist. The Super Admin tier is largely inert (see §5.1).
4.3 Token carries identity and tenant	✅	<code>LoginUseCase.ts:47-53</code> — payload has <code>userId</code> , <code>role</code> , <code>tenantId</code> , <code>tenantSlug</code> , <code>warehouseId</code>	
4.4 Role-based permissions	●	<code>authorize()</code> on most routers	The quotation router has no <code>authorize()</code> at all; gating is pushed into use cases inconsistently.
4.5 No cross-tenant data access	✅	<code>resolveTenant.ts:29-31</code> ; integration tests assert 403	Verified by <code>tests/integration/dashboard/dashboardRoutes.test.ts</code> .
4.6 Cloud deployment	⚠	CORS allow-lists a Vercel host; uploads served from local disk via <code>express.static</code> (<code>app.ts:295</code>)	SRS §10 specifies S3-compatible object storage. Local disk does not survive a container restart and cannot scale horizontally.
4.7 Horizontal scalability	●	Stateless JWT auth is scale-friendly	Blocked by local-disk media, plus several in-memory aggregations.
4.8 Directory-based tenant URLs	✅	<code>routes/index.tsx:81</code> <code>/:tenantSlug</code> ; <code>app.ts:116-351</code>	Matches spec precisely.

§5 — Roles & Permissions

Requirement	Status	Evidence	Notes
5.1 Super Admin: create tenants	❌	Only <code>GET /api/tenants</code> exists	Tenants are created by public self-registration (<code>POST /api/auth/register</code>), which contradicts the spec.
5.1 Super Admin: suspend / remove tenants	❌	No endpoint; no <code>subscriptionStatus</code> on <code>Tenant</code>	<code>TenantManagementTable.tsx:36-44</code> renders Plan/Status as — placeholders.
5.1 Platform-wide usage metrics	❌	<code>SuperAdminDashboard.tsx:19-20</code> <code>mockGlobalEvents</code> ; two KPI cards read " --- " / "Coming in Phase 5"	Only the tenant count is real.
5.1 Super Admin cannot read tenant business data	⚠	<code>resolveTenant.ts:30</code> explicitly exempts <code>SUPER_ADMIN</code> from the tenant check	Inverts the spec. Most controllers then read a null <code>req.user.tenantId</code> and crash rather than leak — an accident, not a control.

Requirement	Status	Evidence	Notes
5.2 Owner: manage company profile / localization / language	✗	<code>AccountSettingsPage.tsx:143-220</code> — every input carries <code>disabled</code> , comment reads "not yet backed by the API"	
5.2 Owner: invite staff	✓	<code>POST /api/:tenantSlug/auth/invitations</code> + <code>InviteMemberModal.tsx</code>	
5.2 Owner: edit staff	●	<code>PUT /auth/staff/:id</code> → <code>UpdateUserRoleUseCase</code>	Role and warehouse only.
5.2 Owner: remove staff	✗	No delete/deactivate on <code>IUserRepository</code> or <code>AuthController</code>	Only pending invitations can be cancelled. Departing staff cannot be off-boarded.
5.3 Staff scoped to assigned/shared clients (configurable)	✗	<code>SearchClientsUseCase.ts</code> applies no user filter	Staff see all tenant clients; no configuration toggle exists.
5.3 Staff manage own quotations	✓	<code>SearchQuotationsUseCase.ts:23</code> forces <code>createdByUserId</code> for <code>STAFF</code>	Correctly enforced on mutations too.
5.3 Staff cannot access company settings	✓	<code>SettingsController.ts:59-62</code>	
5.4 Matrix: Reports "Limited / personal" for Staff	✗	Reports router is <code>authorize(['BUSINESS_OWNER', 'SUPER_ADMIN'])</code> ; React route is <code>RoleGuard</code> -blocked	Staff get zero reports. Worse, <code>useNavigation.ts:27</code> still shows Staff a Reports link — clicking it lands on <code>/unauthorized</code> .

§6.1 — Dashboard

Requirement	Status	Evidence	Notes
Cards: total clients, upcoming appointments, low stock	✓	<code>GetTenantClientMetricsUseCase.ts</code> computes all three from real data	Genuinely wired.
Quick links to recently viewed clients	✗	No "recently viewed" tracking anywhere	An activity feed exists, which is a different thing.
Quick links to pending quotations	✗	Owner dashboard shows a count only; no link or list	
Calendar widget on dashboard	✓	<code>UpcomingAppointmentsWidget.tsx</code> → <code>GET /appointments/upcoming</code>	
Owner sees company-wide KPIs	✓	<code>BusinessOwnerDashboard.tsx</code>	
Staff see personal / assigned data only	✗	<code>/dashboard/metrics</code> is unscoped by user; <code>/feed</code> never passes <code>userId</code> despite the use case accepting it (<code>dashboardRoutes.ts:47-53</code>)	Staff see tenant-wide client counts, quotation counts and stock alerts. Direct violation of §6.1 and §5.4.
Super Admin platform dashboard	●	Real tenant list; everything else mocked	
KPI visibility follows record permissions	✗	See above	

\$6.2 — Client Management

Requirement	Status	Evidence	Notes
Add client: name, contact, company	●	<code>ClientFormContent.tsx:127–153</code> — name, email, phone, status	No company field on the <code>Client</code> model or form.
Assign client to a staff member	✗	<code>Client.assignedUserId</code> exists in schema and API, but no assignment control exists in the form	The field can never be populated through the UI.
Tag with categories (prospect / active / inactive)	✓	<code>ClientStatus</code> enum + <code>SelectInput</code>	
Edit client	✓	<code>PUT /clients/:clientId</code> → <code>UpdateClientUseCase</code>	
Record who last updated	✓	<code>Client.lastUpdatedByUserId</code>	Stored; not surfaced in the UI.
Custom fields: owner-defined TEXT / NUMBER / DATE / SINGLE_SELECT	●	<code>DefineCustomFieldUseCase.ts</code> , owner-gated	UI/API mismatch (bug 1): the form offers <code>BOOLEAN</code> , not a valid <code>FieldType</code> , so the request 400s. <code>SINGLE_SELECT</code> cannot be created at all. No edit or delete.
Custom fields on Add / Edit / Search	●	Present on Add and Edit (<code>ClientFormContent.tsx:168</code>)	Backend supports custom-field search via JSONB <code>@></code> , but no UI exposes it.
Client history: unified timeline	●	<code>GetClientHistoryUseCase.ts</code> merges interactions + appointments	Quotations are absent from the timeline — explicitly required by §6.2.
Outcome categories, configurable per tenant	✓	<code>OutcomeCategory</code> model + <code>DefineOutcomeCategoryUseCase</code> + UI tab	Create only; no edit or delete.
Status changes over time, who and when	✗	No <code>ClientStatusHistory</code> ; appointment status changes are not logged either	Only the latest editor is stored.
Search by name	✓	<code>ClientListContent.tsx:33</code>	Debounced, 300 ms.
Search by contact number or email	✗	<code>SearchClientsFilters</code> has no email or phone key	Explicitly required by the SRS.
Filter by assigned staff / status	●	Backend supports both	Neither filter is rendered in the UI.
Notes: timestamped, chronological, feeding the timeline	✓	<code>Interaction</code> with <code>channel='NOTE'</code>	Well done.

\$6.3 — Appointments

Requirement	Status	Evidence	Notes
Daily / weekly / monthly calendar	✓	<code>CalendarContent.tsx:45</code> <code>viewMode: 'day' 'week' 'month'</code>	
Colour-coded statuses	✓	Badge variant per status	
Dashboard widget parity	✓	Same <code>/appointments/upcoming</code> source	
Select client, date, time, service/purpose	●	<code>createAppointmentSchema</code> — <code>clientId</code> , <code>assignedUserId</code> , <code>scheduledAt</code> , <code>notes</code>	No dedicated service/purpose field; <code>notes</code> is a free-text stand-in.
Assign to a staff member	✓	<code>assignedUserId</code> , required, tenant-validated	

Requirement	Status	Evidence	Notes
Create from calendar <i>and</i> from client profile	✓	<code>CreateAppointmentPage.tsx</code> and <code>ClientDetailContent.tsx:19</code> both use <code>AppointmentForm</code>	Exactly as specified.
Statuses Scheduled / Confirmed / Completed / Cancelled	✓	<code>Appointment.ts</code> with terminal-state guards	<code>complete()</code> correctly requires <code>CONFIRMED</code> first.
Reschedule preserving history	✓	<code>AppointmentAuditLog</code> rows written per reschedule	
Notify assigned staff of reschedule	✗	No notification code path exists	
Cancel with <i>optional</i> reason	⚠	<code>cancelAppointmentSchema</code> makes reason required; <code>Appointment.cancel()</code> then discards it (<code>Appointment.ts:103</code>)	Wrong in both directions: over-validated, then thrown away.
Cancelled appointments retained in history	✓	Row persists with <code>CANCELLED</code>	
Full audit trail of every status change	✗	<code>AppointmentAuditLog</code> models only <code>previousDate</code> / <code>newDate</code>	It cannot represent a status change. Only reschedules are audited.

\$6.4 — Products & Stock

Requirement	Status	Evidence	Notes
Product/service records: name, description, price	✓	<code>Product</code> model (plus sku, brand, cost, tags, images)	Exceeds spec.
Full inventory list	✓	<code>InventoryList.tsx</code> , paginated and faceted	
Multiple warehouses, owner-managed	✓	<code>Warehouse</code> model, owner-gated CRUD	
Stock tracked per location	✓	<code>StockLevel</code> unique on (<code>tenantId</code> , <code>productId</code> , <code>warehouseId</code>)	
Per-location and company-wide totals	✓	<code>GetProductStockBreakdownUseCase</code> ; dashboard sums across warehouses	
Low-stock alerts evaluated per location	⚠	<code>lowStockThreshold</code> is a product-level field; the dashboard sums across warehouses before comparing (<code>GetTenantClientMetricsUseCase.ts:96-97</code>)	\$6.4 requires per-location evaluation "so a business can see exactly where a shortage exists". Search-level availability is per-warehouse; the alert is not.
Categories: create, assign, filter	✓	<code>Category</code> model + owner-gated CRUD + search facet	
Manual adjustment with recorded reason, at a location	✓	<code>AdjustStockUseCase.ts</code> — writes a <code>StockMovement</code> , blocks negative stock	
Transfer between locations, both sides recorded	✓	<code>TransferStockUseCase.ts</code> , inside a DB transaction	Model implementation.

Requirement	Status	Evidence	Notes
Mark Available / Unavailable per location	⚠️	Availability is derived from quantity (<code>IN_STOCK</code> / <code>LOW_STOCK</code> / <code>OUT_OF_STOCK</code>), never manually set	Reasonable design, but a service with no stock rows can never be marked "Available" — a real gap for the service-business persona.
Flag low-stock on dashboard	✅	Owner dashboard "Inventory alerts" card	

\$6.5 — Quotations

Requirement	Status	Evidence	Notes
Select client, add products with qty/price from a warehouse	✅	<code>CreateQuotationUseCase.ts</code> validates client, product and warehouse all belong to the tenant	
Automatic subtotal	✅	<code>Quotation.ts:45-47</code> computed getter	Correctly derived, not stored.
Generate a shareable / viewable version	❌	<code>QuotationDetailContent.tsx:122</code> — <code>onClick={() => alert('PDF generation coming soon')}</code>	No PDF, no public link, no email.
Mark as Sent	⚠️	<code>submit / approve</code> set <code>status=SENT</code> , <code>sentAt=now()</code>	Status flips but nothing is delivered to the client. "Sent" is currently a fiction.
Six statuses	✅	<code>Quotation.ts:3-10</code>	Exact match to spec.
Filter by status or client	🟡	Backend <code>SearchQuotationsUseCase</code> supports both	Frontend filters the current page in memory (<code>QuotationListContent.tsx:54-61</code>) — tab counts and pagination are wrong past page 1.
Status history retained	✅	<code>QuotationStatusHistory</code> written on every transition	
Owner can enable / disable approval step	✅	<code>Tenant.requiresQuotationApproval</code> + toggle + <code>SettingsService</code>	
Staff quotation → Pending Approval when enabled	✅	<code>Quotation.submit()</code>	
Direct Draft → Sent when disabled	✅	Same method	
Who is authorised to approve is configurable	❌	Hard-coded to <code>BUSINESS_OWNER</code> in <code>ApproveQuotationUseCase</code> / <code>GetPendingApprovalsUseCase</code>	\$6.5 requires this be configured per business.
No stock reserved in Draft / Pending	✅	No stock code on those paths	
Stock deducted on Accepted	✅	<code>MarkQuotationAcceptedUseCase.ts:36-73</code> — pre-validates, then deducts transactionally and logs movements	Best-implemented rule in the codebase.
Reversal via manual stock adjustment	✅	<code>AdjustStockUseCase</code> covers it	

Requirement	Status	Evidence	Notes
Expiry	⚠️	<code>POST /:id/expire</code> exists but is manual only	No scheduled job; quotations never expire on their own.

§6.6 — Notifications

Requirement	Status	Evidence
Email notifications on key events	✗	<code>ISender</code> exposes only <code>sendPasswordResetEmail</code> and <code>sendInvitationEmail</code> ; <code>grep emailSender\.</code> finds exactly two call sites, both in auth.
Owner configures which events email which roles	✗	No configuration model, UI or endpoint.
Appointment reminders	✗	No scheduler, cron or queue anywhere in the repository.
In-platform reminder visibility	✗	No <code>Notification</code> model; <code>grep -ril notification backend/src</code> returns zero files.
Configurable reminder lead time	✗	—
Follow-up reminders on unanswered quotations	✗	—
Configurable follow-up waiting period	✗	—

Module completion: 0%. This is Milestone 6 in its entirety.

§6.7 — Reports

Requirement	Status	Evidence	Notes
Total clients broken down by status	✅	<code>getClientStatusDistribution</code> → donut chart	
Trend of new clients over a period	✗	No time-series client query	
Appointment statistics by status	✗	No appointment report use case, endpoint or chart	Named MVP deliverable.
Appointment breakdown by staff / date range	✗	—	
Total products and stock levels, by warehouse	✅	<code>getInventoryValueByWarehouse</code> → two bar charts	
List of low-stock or unavailable items	✗	Reports page shows aggregate value and count only	
Reports scoped to viewing company	✅	All queries take <code>tenantId</code>	
Owner can configure which widgets show	✗	Four charts hard-coded in <code>ReportsPage.tsx</code>	
Staff see a permission-limited version	✗	Staff are blocked entirely (see §5.4)	

Requirement	Status	Evidence	Notes
<i>Extra:</i> monthly revenue chart	⚠️	Not in the SRS; also filters on a non-existent 'WON' status (PrismaReportRepository.ts:17)	Dead filter value.

§6.8 — Settings

Requirement	Status	Evidence	Notes
Edit company name, address, contact details	❌	Tenant has only name , urlSlug , logoUrl , coverImageUrl , requiresQuotationApproval ; fields rendered disabled at AccountSettingsPage.tsx:151-166	Company name is set once at registration and never editable.
Company logo	✅	ImagePicker → /api/:tenantSlug/media , sharp-processed	Working.
Manage staff list	●	TeamSettingsContent.tsx — list, invite, change role, cancel invite	No removal.
Date / time / number formatting per company	❌	Disabled UI at AccountSettingsPage.tsx:171-204 ; no persistence	
Default currency for quotations and reports	❌	Hard-coded currency: 'USD' at ReportsPage.tsx:29-33	
User-level language switching	❌	No language field on User ; UI disabled	
Translation layer, no hard-coded strings	❌	No i18n library in frontend/package.json ; every string is a literal	The SRS calls this out as an architectural requirement. register accepts a locale param that is silently discarded.

§7 — Mobile / Responsive · §8 — Security

Requirement	Status	Evidence	Notes
7. Responsive across desktop / tablet / mobile	●	52 CSS modules contain @media ; DataTable has a card-mode fallback; dashboards branch mobile vs desktop	Not verifiable at real breakpoints without running the app. Mobile Staff dashboard tiles are fake data.
8.1 Credential login + session tokens	●	JWT with httpOnly + secure + sameSite: 'strict' (AuthController.ts:49-52)	Good cookie hygiene, undermined by also writing the token to localStorage (useLogin.ts:16) and sending it as a Bearer header. Any XSS recovers the token.
8.1 Sessions expire after inactivity	❌	Fixed JWT_EXPIRATION '24h'	No refresh token, no idle timeout, no revocation.
8.2 Strong one-way password hashing	✅	bcrypt, 10 rounds (BcryptPasswordHasher.ts)	
8.3 RBAC on every action	●	Middleware on most routers	Quotation and settings routers rely purely on in-use-case checks; POST /inventory/products is open to any Staff member.

Requirement	Status	Evidence	Notes
8.4 Authenticated APIs validating identity + tenant	✓	<code>authenticate</code> + <code>resolveTenant</code> on all tenant routers	
8.5 Tenant isolation	✓	Verified in code and integration tests	
8.6 HTTPS	⚠	Deployment concern; no HSTS, no redirect middleware, no <code>helmet</code>	
<i>implied</i> Safe error handling	✗	<code>app.ts:353-356</code> returns <code>err.message</code> and <code>err.stack</code> to the client on every 500	Leaks internals in production.
<i>implied</i> Secret management	✗	<code>process.env.JWT_SECRET 'secret'</code> (<code>JwtTokenService.ts:8</code>)	Silently boots with a guessable secret — anyone can forge a Super Admin token.
<i>implied</i> Brute-force protection	✗	No rate limiting on <code>/login</code> or <code>/password-reset</code>	

3. Missing Features by Priority

Critical — blocks MVP sign-off

1. **Entire Notifications module (\$6.6)** — email event notifications, appointment reminders, follow-up reminders, plus per-business configuration for all three. Requires a `Notification` model, a scheduler and settings.
2. **Multi-language / i18n layer (\$6.8, \$12.1)** — no translation infrastructure; retrofitting means touching every component.
3. **Company profile & localization settings (\$6.8)** — company name, address and contact are not editable; date, number and currency formats are not configurable.
4. **Appointment statistics report (\$6.7)** — a named MVP deliverable with no backend or frontend.
5. **Default JWT secret (\$8)** — must fail fast when `JWT_SECRET` is unset.
6. **Stack traces in HTTP 500 responses (\$8)** — `app.ts:355`.

High

7. Role-scoped dashboard data (\$6.1, \$5.4) — Staff currently see company-wide KPIs.
8. Client → staff assignment UI (\$6.2) — the field exists but is unreachable.
9. Client search by email and phone (\$6.2) — explicitly specified.
10. Staff account removal (\$5.2).
11. Quotation delivery — PDF or share link (\$6.5); "Sent" currently delivers nothing.
12. Super Admin tenant provisioning: create / suspend / remove (\$5.1).
13. Reports for Staff (\$5.4) — currently 403 while the nav link is still shown.
14. Remove the JWT from `localStorage` (\$8); rely on the `httpOnly` cookie.
15. Rate limiting on auth endpoints plus `helmet` (\$8).

Medium

16. Appointment status-change audit trail (\$6.3).
17. Quotations in the client history timeline (\$6.2).
18. Client status-change history with actor and timestamp (\$6.2).
19. Configurable quotation approver (\$6.5).
20. Low-stock / unavailable item list in Reports (\$6.7).
21. New-client trend chart (\$6.7).
22. Per-location low-stock evaluation (\$6.4).
23. Automated quotation expiry job (\$6.5).
24. Custom field edit/delete plus `SINGLE_SELECT` support in the UI (\$6.2).
25. Server-side quotation filtering in the list UI (\$6.5).
26. Dashboard quick links: recently viewed clients, pending quotations (\$6.1).
27. Object storage for media instead of local disk (\$10, \$4.7).

Low

28. Client "company" field (\$6.2).
29. Dedicated appointment service/purpose field (\$6.3).
30. Configurable report widgets (\$6.7).
31. Outcome category edit and delete (\$6.2).
32. Manual availability override for service-type products (\$6.4).
33. Remove the out-of-scope Revenue report, or add it to the specification.

4. Bugs & Deviations

#	Severity	Finding
1	High	Custom field type mismatch. <code>ClientSettingsContent.tsx:196</code> offers <code>BOOLEAN</code> , which is not in <code>FieldType</code> (<code>TEXT</code> , <code>NUMBER</code> , <code>DATE</code> , <code>SINGLE_SELECT</code>). <code>z.nativeEnum(FieldType)</code> rejects it → 400 with no user-facing message. Conversely <code>SINGLE_SELECT</code> is unreachable from the UI because there is no options editor, and the domain requires at least one option.
2	High	Staff "Reports" nav link leads to 403. <code>useNavigation.ts:27</code> pushes a Reports item for warehouse-assigned Staff; the route is <code>RoleGuard</code> -restricted to owners, so it lands on <code>/unauthorized</code> . The same block pushes a duplicate <code>inventory</code> item, producing two identical sidebar entries and duplicate React keys.
3	High	Dead "My Tasks" nav item. <code>useNavigation.ts:19</code> — no <code>tasks</code> route exists; Staff clicking it hit the 404 page.
4	High	Fabricated data on the Staff dashboard. <code>StaffDashboard.tsx</code> hard-codes "My assigned clients" = 156, REVENUE \$12.4k, CLOSED 18, WIN RATE 64%, and a static "Today, Oct 24" button. The mobile Quick Action buttons have no <code>onClick</code> . A user cannot distinguish these from real figures.
5	High	Staff dashboard KPIs are tenant-wide. <code>useDashboardMetrics()</code> calls the unscoped <code>/dashboard/metrics</code> , so the Staff card labelled "New clients" is the whole company's client count.
6	Medium	Cancellation reason discarded. The API requires a non-empty <code>reason</code> ; <code>Appointment.cancel()</code> accepts and drops it. The SRS wants it optional <i>and</i> retained.
7	Medium	tenantId sourced inconsistently. Client and appointment controllers use <code>req.tenant.id</code> (URL-resolved); inventory (23×) and quotations (11×) use <code>req.user!.tenantId!</code> (token). For a <code>SUPER_ADMIN</code> , whom <code>resolveTenant</code> deliberately lets through, that value is <code>null</code> and the non-null assertion produces a 500 or a silent empty result.
8	Medium	Assigned user rendered as a raw UUID. <code>ClientListContent.tsx:89</code> and <code>ClientDetailContent.tsx:162</code> display <code>client.assignedUserId</code> where a name belongs.
9	Medium	Quotation list filtering is client-side (<code>QuotationListContent.tsx:54–61</code>) despite backend support — tab counts are wrong and filters miss records beyond the current page.
10	Medium	Dead status value in revenue report. <code>PrismaReportRepository.ts:17</code> filters <code>status IN ('ACCEPTED', 'WON')</code> ; <code>'WON'</code> is not in <code>QuotationStatus</code> .
11	Medium	locale accepted then dropped on registration (<code>authSchemas.ts:9</code> vs <code>RegisterBusinessOwnerUseCase</code>) — no <code>locale</code> column exists.
12	Medium	Super Admin bypasses tenant scoping in <code>resolveTenant.ts:30</code> — the inverse of the \$5.1 privacy guarantee.
13	Low	No duplicate-email check on registration — only slug uniqueness; <code>User</code> has no unique constraint on <code>(tenantId, email)</code> in the schema.
14	Low	Reports currency hard-coded to USD (<code>ReportsPage.tsx:31</code>) against the \$6.8 per-company currency requirement.
15	Low	Unaddressed developer question shipped in a route handler — <code>dashboardRoutes.ts:47–51</code> contains a stream-of-consciousness comment ("Staff see only their activity feed? ... Wait, let's check the usecase.") where the scoping decision should be.

5. Code Quality Observations

Strengths. The hexagonal layering is real and consistently applied — domain entities own their invariants (`Quotation` refuses illegal transitions, `StockLevel` refuses negative quantities, `Appointment` guards terminal states), use cases are thin and testable, and infrastructure sits behind ports. Multi-step stock operations correctly use an `IStockTransactionManager`. Several comments explain *why* rather than *what*, which is unusually good. This is a codebase worth extending.

Concerns

1. **The test suite is red and non-deterministic.** `npx jest tests` fails 13 suites / 29 tests in parallel but only 5 suites / 1 test serially — integration tests share one database with no per-worker schema isolation. Four of the serial failures are stale test doubles that no longer compile (`InMemoryUserRepository` missing `updateRoleAndWarehouse`, `IInvitationRepository` missing `delete`, `req.user` missing `warehouseId`); one is a genuine failure in `RequestPasswordResetUseCase.test.ts`. Suites that do not compile do not run — and the ones going dark include `resolveTenant`, the client routes and the inventory routes, exactly the tenant-isolation guarantees you most want covered.
2. **createApp is a 360-line service locator** (`app.ts`) mixing ES `import` with 40+ inline `require()` calls and instantiating around 60 objects by hand. Adding a module means editing this function, and the `require()` calls defeat type checking at every wiring point.
3. **In-memory aggregation on unbounded sets.** `GetTenantClientMetricsUseCase.ts:52` fetches up to 10,000 clients into memory on every dashboard load to count one week's worth, with a comment acknowledging it. `getMonthlyRevenue` pulls 12 months of quotations with all line items and reduces in JS. Both should be SQL aggregates. This directly threatens §4.7.
4. **Media on local disk.** `express.static(UPLOADS_DIR)` (`app.ts:295`) prevents multi-instance deployment and loses uploads on redeploy. §10 specifies S3-compatible storage.
5. **Error handling is string-matching.** Controllers branch on `error.message.includes('Unauthorized')` / `.includes('not found')` — around 20 sites in `QuotationsController` alone. Typed domain errors exist but are not used consistently. Renaming a message silently changes an HTTP status code.
6. **Weak typing at the boundary.** `authorRole: req.user!.role as any` appears 24× in `inventoryController.ts`; `LoginUseCase.execute(input: any); integrationRepository?: any`. Non-null assertions on `req.user!.tenantId!` are how bug 7 becomes a runtime 500.
7. **No API documentation and no observability** — no OpenAPI/Swagger, no structured logger, no health endpoint, no metrics. The §4.6 "monitoring" expectation is unaddressed.
8. **Presentation-layer leakage into infrastructure.** `SettingsController.getSettings` calls `require('prisma')` inline and queries directly, bypassing the repository it was handed (`SettingsController.ts:29–34`).
9. **Accessibility is thin.** 32 `aria-*` / `role` attributes across the whole UI library; the Storybook a11y addon is installed but there is no evidence it gates anything. Not formally assessed — flagged as *Unable to Verify* pending an axe run.

6. Recommended Actions

Phase A — Correctness & security (before any deployment)

1. Fail fast on a missing `JWT_SECRET`; remove the `'secret'` fallback.
2. Replace the global error handler with one that logs internally and returns `{ error: 'Internal server error' }` — never `err.stack`.
3. Remove the `localStorage` token and the Bearer interceptor; rely on the `httpOnly` cookie.
4. Add `helmet` and `express-rate-limit` on `/login`, `/password-reset/request` and `/register`.
5. Fix the four non-compiling test suites and the `RequestPasswordResetUseCase` failure; give integration tests per-worker DB isolation (or `maxWorkers: 1`) so `npm test` is green and deterministic.
6. Fix bugs 1, 2 and 3 (custom-field `BOOLEAN`, Staff Reports 403 link, dead "My Tasks") — all small, all user-visible.
7. Standardise `tenantId` on `req.tenant.id` across every controller (bug 7) and decide the Super Admin data-access policy deliberately (bug 12).

Phase B — Close the named MVP gaps

8. **Build the Notifications module:** a `Notification` model plus per-tenant `NotificationSettings`; a scheduled worker for appointment reminders and quotation follow-ups; event-driven emails on appointment created/rescheduled and quotation responded; an in-app notification list; and a configuration UI. *Largest single item — budget accordingly.*
9. **Introduce i18n:** adopt `react-i18next`, externalise all strings, add a `language` column on `User` and a language switcher. Costly to defer further — every new screen widens the gap.
10. **Company profile & localization:** extend `Tenant` with `address`, `contactEmail`, `contactPhone`, `locale`, `timezone`, `dateFormat`, `currency`; wire the already-built (disabled) settings form; consume `currency` in Reports and quotations.
11. **Appointment statistics report** — counts by status, breakdown by staff and date range, plus the client-growth trend and the low-stock / unavailable list.
12. **Role-scope the dashboard:** pass `userId` for `STAFF` to both `/metrics` and `/feed`; delete every hard-coded figure from `StaffDashboard` and `SuperAdminDashboard`, or label them unmistakably as placeholders.
13. **Quotation delivery:** server-side PDF plus a tokenised public view URL, sent on transition to `SENT`.
14. **Super Admin tenant management:** `subscriptionStatus` on `Tenant`; create / suspend / remove endpoints; wire `TenantManagementTable`.
15. **Client assignment:** add a staff selector to the client form, resolve `assignedUserId` to a display name, and add the assigned-staff and status filters to the list.
16. **Client search by email and phone.**
17. **Staff removal / deactivation.**

Phase C — Fidelity & hardening

18. Add quotations to the client history timeline; add `ClientStatusHistory` and appointment status-change audit rows.
19. Make the quotation approver role configurable per tenant.
20. Move `lowStockThreshold` evaluation to per-warehouse.
21. Add a scheduled job for automatic quotation expiry.
22. Move dashboard and report aggregation into SQL; drop the 10,000-row fetch.
23. Migrate media to S3-compatible object storage.
24. Server-side quotation list filtering and pagination.

25. Introduce typed domain errors with an error-to-status mapper; retire `message.includes(...)` branching.
26. Split `createApp` into per-module composition functions with real `import s`.
27. Add OpenAPI docs, structured logging and a `/health` endpoint.
28. Run an axe / Lighthouse pass and a genuine breakpoint audit to close out §7 and the accessibility NFR — both are currently *Unable to Verify* from source alone.

Scope caveats. Responsive behaviour (§7) and accessibility were assessed from source only; confirming them requires running the application at real breakpoints with an automated accessibility checker, which was not performed. HTTPS and cloud-deployment items (§4.6, §8.6) are infrastructure concerns with no artifacts in this repository — they are marked 🚩 rather than ❌ since they may be handled at the hosting layer.